

Wed, Apr 8 at 2:02 PM

I am working on a Python script to create a list of physically correct materials based on Material X and Carma in Houdini. This is a part of a big Neuron project. Here is the project brief:

```
"""
```

```
# PROMPT: Implementation of "Neuron" - Latent Scene Engine
```

```
## Context & Vision
```

```
I am a 3D artist and pipeline developer building "Neuron,"
```

Show more ▾



I'm auditing this in three layers: physical parameter logic, Houdini/MaterialX wiring, and data semantics for training. First pass already shows a few hard bugs in the material builder, not just tuning issues.

The biggest hard bug is already clear: your colorable materials do not actually use the chosen palette color in the shader. You write `params["color"]` , but the builder only reads `base_color` , so `car_paint_red_*` and similar entries will render with the base's default color, not the selected one.

Thought for 3m 2s >

The script is a decent **material metadata generator**.

It is **not yet a physically correct material builder**.

The main problem is simple: your JSON contains a lot of material intelligence, but the Houdini builder only consumes a tiny subset of it. Most of the interesting parameters are dead.

What is critically wrong

1) Colorable materials are broken

This is a hard bug.

In `_build_entity()` you do this:

</> Python



▶ Run

```
if color_override:
    params["color"] = color_override
```

But your builder only maps:

</> Python



▶ Run

```
PARAM_MAP = {
    "base_color": "base_color",
    "metalness": "metalness",
    "specular_ior": "specular_IOR",
    "specular_roughness": "specular_roughness",
    "specular_anisotropy": "specular_anisotropy",
}
```

So `color` is ignored completely.

Result:

- `car_paint_red_matte_dusty` will not use the red override
- all future `colorable` materials are wrong

This should be:

</> Python



▶ Run

```
if color_override:
    params["base_color"] = color_override
```

That is the first fix.

2) Most of your material system does nothing

This is the biggest architectural failure.

Your JSON generates these parameters:

- `transmission`
- `transmission_color`
- `transmission_depth`
- `transmission_scatter`
- `transmission_dispersion`
- `thin_walled`
- `coat`
- `clearcoat_weight`
- `coat_roughness`
- `subsurface`

- sss_color
- sheen
- bump_scale
- bump_type
- noise_scale
- dirt
- wear
- variation_seed
- metallic_flake

But the shader builder ignores all of them.

That means:

- glass is not actually glass
- honey is not actually volumetric/translucent
- car paint has no real coat layer
- scratched / dusty / hammered / brushed logic is mostly fake
- your Copernicus "Big Four" design is not implemented at all

Right now the material network is basically:

- base color
- metalness
- IOR
- roughness
- anisotropy

That is nowhere near the brief.

3) Transparent materials will render wrong

`glass_polished_clean`, `glass_matte_clean`, and `honey_satin_dusty` are fundamentally broken in the current builder.

You correctly store transmission-related values in JSON, but you never apply them to `mtlxstandard_surface`.

According to the Houdini MaterialX Standard Surface docs, transmission, transmission color, transmission depth, transmission dispersion, and transmission roughness are distinct parameters of the shader. Thin-walled behavior is also special-cased.  SideFX +1

So with the current code:

- `glass_*` will not become transparent
- `honey_*` will not behave as intended
- your “translucent” category is mostly decorative metadata

4) Honey’s volumetric logic is not valid for Karma the way you are using it

You set:

 Python



 Run

```
"transmission_scatter": [0.8, 0.5, 0.1]
```

for honey.

Problem: Houdini’s `mtlxstandard_surface` page explicitly says **Transmission Scatter is not supported by Karma**.  SideFX

So even if you mapped that parameter, Karma would not give you the result you think.

For honey, wax, ice, milky liquids, frosted transmissives:

- either use supported subsurface controls
- or move to a proper volume/absorption setup
- but do not assume `transmission_scatter` will solve it in Karma

5) Your “physically correct metal” claim is not true yet

You store conductor-style values:

Python

```
"specular_ior": ...  
"k": ...
```



Run

But:

- `k` is never used
- your builder only sets IOR
- there is no direct mapping in your code from complex IOR data to the actual shader response

The SideFX Standard Surface docs expose Metalness, Specular Color, and Index of Refraction, but not a direct `k` parameter in the node interface you are using.  SideFX +1

So your metal library is currently:

- good metadata
- not physically complete shader implementation

For training labels, that may be acceptable early.
For "physically correct material library," it is not.

6) Your Copernicus HDA pipeline is not wired in at all

Your brief says the whole system depends on these outputs:

- Variation Mask
- Dirt Mask
- Wear Mask
- Micro-Bump

and then those should drive MaterialX mix / multiply / normal logic.

Your current `_create_builder()` creates only:

- `mtlxstandard_surface`
- `mtlxdisplacement`
- subnet outputs

No Copernicus HDA instance.

No mix nodes.

No multiply nodes.

No normal map chain.

No dirt/wear/variation wiring.

So:

- `variation_seed`
- `dirt`
- `wear`

- bump_type
- noise_scale
- bump_scale

all exist in JSON but have **zero effect** on the shader.

That is not a small missing feature. That is the core of the Neuron shading system, and it is absent.

7) thin_walled logic is wrong source-of-truth

This:

 Python



 Run

```
if any(kw in b_name for kw in ("bubble", "window")):  
    params["thin_walled"] = True
```

is dead or nearly dead logic.

Your base names are things like:

- glass
- water
- ice
- honey

So this almost never triggers.

Also, thin-walled is not a material identity issue. It is mostly:

- geometry thickness
- asset intent
- shot context

The docs also make clear that thin-walled transmission behaves differently and disables refraction.  SideFX

So this should be an explicit authored flag, not inferred from a substring in the base name.

8) Prompt generation has grammar defects that will hurt training quality

Examples from your generated labels:

- “with a **directional micro-scratches** surface”
- “with a **indented with small craters** surface”

That is bad English, and bad prompt quality.

Your template system assumes `finish` is an adjective phrase, but many finish hints are noun-style descriptions.

This will weaken CLIP-side supervision.

Not catastrophic, but worth fixing early.

What is physically suspicious, even where the code “works”

These are not hard bugs, but they are weak spots.

Polished rubber roughness = 0.02

That is too mirror-like for most rubber.

Rubber can be glossy, but 0.02 reads like lacquered hard plastic or coated surface, not ordinary rubber.

For a "black polished scratched rubber" stress test, something like 0.12-0.25 is more believable unless this is intentionally wet/oiled/coated.

Default translucent dispersion = 1.0

You assign:

</> Python



▶ Run

```
"transmission_dispersion": 1.0
```

at the translucent category level.

That means glass, water, honey, ice all inherit dispersion by default.

That is too broad.

For most production training data:

- water: effectively near zero unless you are explicitly chasing prism effects
- common glass: usually subtle enough to ignore
- gemstones: yes
- diamond/ruby/emerald: yes

So dispersion should be opt-in, not category default.

The shader supports transmission dispersion as a distinct parameter.  SideFX

hint **pollutes** parameters

Because of merge order:

</> Python



▶ Run

```
params = {**defaults, **category, **base, **finish, **cond}
```

every layer's hint can overwrite the previous one, and in practice the condition hint wins.

So your final JSON stores:

</> JSON



```
"hint": "pristine condition"
```

inside render parameters.

That is junk in the shader payload.

It should stay in semantic metadata only.

Structural improvements

1) Split your schema into three clean domains

Right now parameters is a dump of everything.

You need:

</> Python



▶ Run

```
{  
  "shader_parameters": {...},  
  "procedural_parameters": {...},  
  "metadata": {...}  
}
```

Suggested split:

shader_parameters

Things that go directly to MaterialX:

- base_color
- metalness
- specular_ior
- specular_roughness
- specular_anisotropy
- transmission
- transmission_color
- transmission_depth
- transmission_dispersion
- subsurface
- sss_color
- sheen
- coat
- coat_roughness
- thin_walled

procedural_parameters

Things that belong to the Neuron HDA:

- variation_seed
- noise_scale
- bump_scale
- bump_type
- dirt
- wear

metadata

Human-readable / dataset stuff:

- base
- finish
- condition
- semantic_hints
- semantic_label

That one change will clean up the entire system.

2) Fix colorable materials now

Minimal fix:

</> Python



▶ Run

```
if color_override:
    params["base_color"] = color_override
```

Not params["color"] .

3) Expand the actual MaterialX mapping

Your builder needs to apply more than five parameters.

At minimum, map:

- `transmission`
- `transmission_color`
- `transmission_depth`
- `transmission_dispersion`
- `thin_walled`
- `coat`
- `coat_roughness`
- `subsurface`
- `subsurface_color` or your chosen SSS mapping
- `sheen`

Otherwise the library is lying about what it created.

4) Implement the actual HDA wiring

This is the real Neuron step.

Your builder should:

1. create the Copernicus HDA inside each material
2. push:
 - `variation_seed`
 - `noise_scale`

- bump_scale
- bump_type
- dirt
- wear

3. wire outputs into MaterialX:

- Variation → color mix
- Dirt → color mix + roughness lift
- Wear → roughness multiplier
- Micro-Bump → normal / bump input

Until that exists, your procedural dataset is fake.

5) Stop using a raw subnet as the long-term material container

SideFX recommends building Karma materials inside a **Karma Material Builder** within a Material Library LOP, and also recommends using **Collect VOP** as the final node when combining surface and displacement.  SideFX

Your raw subnet approach may work for a minimal case, but for a production Neuron library it is the wrong foundation.

6) Add validation rules before writing JSON

You need a small validation pass that catches obvious nonsense.

Examples:

- `transmission > 0` but `base_color` not blacked out for solid transmissives

- `coat > 0` and no coat roughness defined
- `colorable == True` but no final `base_color`
- `bump_type != "none"` but `bump_scale == 0`
- `category == "metal"` and `k` exists but no conductor conversion path
- `thin_walled == True` on honey / concrete / iron

This will save you a lot of garbage renders.

7) Fix prompt phrasing with adjective-safe fields

Do not reuse raw `hint` text as `{finish}` in all templates.

Store two forms:

`</>` Python



▶ Run

```
"finish_label": "brushed",  
"finish_description": "directional micro-scratches"
```

Then use templates like:

- "a brushed iron surface with directional micro-scratches"
- "industrial concrete with small crater-like hammering"

That will produce much better CLIP supervision.

What I would fix first, in order

Phase A — hard bugs

1. change `color` → `base_color`

2. remove `hint` from shader params
3. split shader params from procedural params
4. map transmission / coat / subsurface / sheen in the builder

Phase B — actual Neuron lookdev

5. instantiate the Copernicus HDA
6. wire Big Four outputs into MaterialX nodes
7. make `wear`, `dirt`, `bump_type`, `variation_seed` actually visible in renders

Phase C — physics cleanup

8. stop defaulting dispersion for all translucent materials
9. revise rubber and a few finish roughness values
10. define a real conductor strategy instead of unused scalar `k`

The blunt verdict

Your current script is **good as a dataset authoring scaffold**.

It is **not good as a physically correct MaterialX/Karma material generator**.

The two most serious issues are:

- **color overrides are broken**
- **most advanced parameters never reach the shader**

Because of that, the generated materials will look much simpler and much less correct than the JSON suggests.

Paste your current Houdini builder network or the `mtlxstandard_surface` parm dump from your exact Houdini version, and I'll map the minimal corrected `_create_builder()` /

_apply_params() for you.



</> Pasted code.py
Python

Here is the updated script. Revise it. I don't like we now having a lot of data in the script root outside of calsses, we will import this module to Houdini tools. If it is fine, ignore this root issue.

```
"""
```

```
"""
```




Pasted text.txt
Document

Here is the data:



Pasted text.txt

Document

Updated data

Thought for 13s >

This brief is based on the updated dataset, where the remaining issues are mostly semantic composition, contamination phrasing, hint consistency, polished micro-breakup policy, and the final alignment of frosted-glass / honey semantics.  Pasted text

</> Markdown



Developer Brief: Final Cleanup Pass for `datagen.materials`

Goal

The data model is now structurally good.

This pass is about removing the last inconsistencies so the generated material records are:

- semantically clean
- non-redundant
- physically intentional
- easy to map into the future Houdini HDA
- stable for CLIP / prompt-conditioned training

This is still a **data-authoring** task.

Do not add Houdini node-building logic here.

1. Main Problems Remaining

The current dataset has improved a lot, but these issues remain:

1.1 Contamination phrasing is duplicated

Some entries describe contamination twice in the final label.

Examples of the pattern:

- finish description already mentions dust on the surface
- condition phrase then repeats dust on the surface again

This affects entries like dusty car paint and dusty honey.

1.2 `semantic\_hints` are less intelligent than `semantic\_label`

The full label logic has improved, but the short hints still use simpler wording.

That creates mismatch between:

- the compact semantic summary
- the final sentence label

1.3 Polished clean materials now use tiny stochastic breakup, but this rule is not clear
This is probably correct, but it needs to be deliberate and documented.

1.4 Frosted glass semantics are much better, but parameter meaning and semantic meaning s
The language now describes blurred transmission through a rough interface.
Make sure the underlying data model is intentionally representing that and not accidentally

1.5 Sticky / viscous contamination wording can be improved
Dust on honey is valid as a stress test, but the phrasing can better express that the contam

2. Fix Semantic Composition Rules

Problem

The generator still builds some labels by stacking:

- finish description
- condition phrase

even when both describe the same visible effect.

That creates repetitive labels.

Required change

Semantic composition must become ****mode-aware****.

Each condition should explicitly declare a semantic mode, and the label composer should deci

- `finish\_description`
- `condition\_phrase`

Do not let both fields say the same thing.

2.1 Define composition policy by condition mode

`pristine`

Meaning:

- the finish description can stay strong
- the condition phrase can stay simple

Use:

- finish carries most of the look
- condition simply confirms clean/pristine state

`abrasion`

Meaning:

- condition modifies the finish appearance directly
- finish wording must soften or adapt

Use:

- finish phrase should mention interruption or damage if needed
- condition phrase should not repeat the exact same scratch concept unless it adds new information

`contamination`

Meaning:

- contamination is a surface layer on top of the base material
- contamination should usually be described once, not twice

Use one of these two patterns only:

Pattern A

- finish stays pure
- condition carries contamination

Example idea:

- finish description: matte surface
- condition phrase: with fine dust settled on the surface

Pattern B

- finish description already includes contamination
- condition phrase becomes very short or neutral

Example idea:

- finish description: matte base with dusty residue
- condition phrase: lightly contaminated at the surface

Do not combine:

- “dusty residue on the surface”
- and “fine dust settled on the surface”

That is redundant.

3. Make `semantic\_hints` Context-Aware

Problem

The final label has smarter phrasing than the hint list.

That weakens the consistency of your semantic payload.

Required change

Generate `semantic\_hints` from the same context-aware logic used for the final label.

Do not build hints by simply concatenating:

- base
- finish adjective
- raw condition phrase

Required behavior

For each material, the short hints should reflect the final intended look.

Examples of desired style

Instead of:

- polished finish
- showing visible scratches and abrasion

Prefer:

- polished finish with visible wear
- surface scratches and abrasion

Instead of:

- matte finish
- with fine dust settled on the surface

Prefer:

- matte finish with surface dust
- fine dust contamination

Instead of:

- satin finish
- with fine dust settled on the surface

Prefer:

- satin finish with adhered surface dust
- sticky contaminated surface

The exact wording can differ.

The important thing is that the hint list must match the final semantic intent.

4. Formalize the Polished Micro-Breakup Rule

Problem

Clean polished materials now have tiny stochastic breakup values.

That is probably good, but the rule is not explicit.

Required change

Decide and document the rule in the generator:

Recommended rule

All real materials may keep a minimum stochastic micro-breakup unless they are intentionally

That means polished does not mean mathematically perfect.

It means:

- very low roughness
- very low bump amplitude
- still a stochastic family if the surface is intended to remain physically believable

Required implementation

Add one explicit policy comment in the generator explaining:

- why polished materials still use `bump\_type: stochastic`
- why very small `bump\_scale` is intentional
- why this helps training by preventing unrealistically perfect surfaces

Optional refinement

Allow a per-base minimum polished bump override.

Examples:

- polished glass may use a different micro-breakup floor than polished gold
- some materials may intentionally go closer to ideal smoothness

This is optional, not mandatory.

5. Improve Contamination Vocabulary for Sticky / Viscous Materials

Problem

Dust on honey is acceptable as a stress test, but the current wording still sounds slightly
For sticky or viscous materials, contamination behaves differently than on hard dry surfaces

Required change

Support condition phrasing that depends on the base material class.

For normal hard-surface contamination

Use phrases like:

- with fine dust settled on the surface
- with surface dust residue
- lightly dust-contaminated surface

For sticky / viscous contamination

Use phrases like:

- with fine dust adhered to the sticky surface
- with surface dust caught in the viscous coating
- with settled dust clinging to the surface

Do not force the same dust phrase onto:

- honey
- glass
- car paint
- rubber

These should not all speak the same semantic language.

Required implementation

Allow condition phrases to be overridden by:

- category
- base
- or a semantic override hook

Recommended design:

- default phrase in condition spec
- optional base-specific semantic override
- optional mode-aware label adjustment

6. Tighten Frosted Glass Intent

Problem

The wording for frosted glass is now good.
Keep it that way.
But make the intent explicit in the generator.

Required change

Define frosted / matte glass as a specific semantic class.

Recommended semantic identity

- translucent
- frosted
- blurred transmission
- rough transmissive interface

Do not drift back toward generic “volumetric scattering” wording unless you intentionally wa

Required implementation

For translucent matte glass, the semantic builder should prefer concepts like:

- frosted glass
- rough transmissive surface
- blurred transmitted light
- diffuse transmission through a rough interface

Keep these as explicit controlled phrases.

Optional improvement

Introduce semantic tags separate from human-readable hints.

For example:

```
- `semantic_tags: ["translucent", "frosted", "blurred_transmission"]`
```

This is optional, but useful later.

7. Strengthen Validation Rules

Problem

The validator is already useful, but it should now validate semantic composition as well as

Required new validation checks

7.1 Redundancy checks

Reject labels or hints when the same contamination phrase is effectively repeated twice.

Examples of bad pattern:

- dusty residue on the surface
- fine dust settled on the surface

within the same label when both clauses mean the same thing.

This does not need to be NLP-heavy.

A simple heuristic is enough.

7.2 Semantic completeness

Require:

- `base\_phrase` not empty
- `finish\_adjective` not empty
- `finish\_description` not empty
- `condition\_phrase` not empty
- `condition\_mode` valid

7.3 Condition mode validation

Allowed condition modes should be fixed and validated.

Recommended set:

- `pristine`
- `abrasion`
- `contamination`
- `aging`

If more modes are needed later, add them deliberately.

7.4 Procedural intent checks

Require:

- `noise\_scale > 0`
- `bump\_scale >= 0`
- `bump\_type` valid
- if `bump\_type == "none"`, `bump\_scale == 0`
- if `bump\_type != "none"`, the chosen type must match the intended finish family

7.5 Semantic alignment checks

For example:

- if hints include `frosted glass`, the label should not describe it as mirror-clear
- if condition mode is `abrasion`, the finish description should not claim pristine mirror p
- if condition mode is `contamination`, the final label should not read as if contamination

These checks can stay lightweight.

The goal is to catch obvious contradictions.

8. Refine the Semantic Data Model

Problem

The current semantic block is already better, but it can be made easier to reason about.

Recommended semantic structure

Keep:

- `base\_phrase`
- `finish\_adjective`
- `finish\_description`
- `condition\_phrase`
- `condition\_mode`
- `semantic\_hints`
- `semantic\_label`

Add, if useful:

- `semantic\_tags`
- `composition\_style`

Example conceptual meaning

`base\_phrase`

What the material fundamentally is.

`finish\_adjective`

Short classifier for UI / metadata / compact prompt use.

`finish\_description`

How the finish visually manifests.

`condition\_phrase`

What happened to the surface state.

`condition\_mode`

How the composer should treat the condition.

`semantic\_hints`

Short retrieval / CLIP helper phrases.

`semantic\_label`

One polished final training sentence.

This structure is good.

Keep it disciplined.

9. Clean Up the Label Templates

Problem

The current labels are readable, but some still feel mechanically assembled.

Required change

Use fewer, stronger template patterns.

Keep the final sentence style simple and controlled.

Recommended template families

Clean / pristine

- close-up of X with Y, in pristine condition
- photorealistic study of X with Y, clean and intact

Abrasion

- study of X with Y disrupted by visible scratches
- close-up of X showing worn surface abrasion over Y

Contamination

- study of X with Y and light surface contamination
- close-up of X with fine dust settled on the surface

Frosted translucent

- close-up of X with a frosted surface that blurs transmitted light
- study of X with rough translucent transmission

Do not generate too many template types.

Consistency matters more than variety.

10. Module / Script Cleanup

The root-level constants are acceptable

Do not waste time rewriting the whole module just because there are many root constants.

That is not the real issue.

Do clean these things

10.1 Remove test subset from global source state

If a debug subset still exists as a module-global list, move it to a function argument.

10.2 Remove legacy / dead code

Delete:

- unused helpers
- deprecated builder logic
- old compatibility remnants that are no longer used

10.3 Keep shared semantic spec neutral

Do not let the data generator depend on prompt-builder internals in an awkward way.

Move shared token / phrase tables to a neutral spec location if needed.

11. Concrete Implementation Tasks

Implement in this order.

Priority 1

1. fix contamination label redundancy
2. make `semantic\_hints` context-aware
3. add semantic validation for repeated or contradictory phrasing

Priority 2

4. formalize polished micro-breakup policy in comments and optional overrides
5. add base-aware contamination phrasing for sticky / viscous materials
6. keep frosted glass semantic vocabulary explicit and controlled

Priority 3

7. refine template families for more natural final labels
8. add optional semantic tags if useful
9. remove any remaining debug / dead code from the module

12. Acceptance Criteria

This revision is done when all of the following are true:

Semantic quality

- dusty materials are not described twice in the same sentence
- hints and labels tell the same story
- worn finishes do not contradict themselves
- frosted glass reads as frosted transmissive material, not something else

Procedural clarity

- polished stochastic micro-breakup is intentional and documented
- procedural fields remain internally consistent

Training quality

- labels are concise, natural, and non-redundant
- contamination reads as a surface condition, not a core identity
- sticky contamination and hard-surface contamination are phrased differently where appropriate

Code cleanliness

- no dead compatibility logic remains
- no debug subset is hardcoded as module-global state
- semantic generation rules are easier to extend later

13. Final Direction

Do not redesign the system again.

The data model is already good enough.

This pass is about tightening the last 10 percent:

- cleaner language
- fewer repeated ideas
- sharper semantic rules
- explicit intent for tiny polished breakup
- better contamination phrasing by material type

That is enough for the next stage.

Sources



I want to start a new chat with you and finish building a "neuromat" Material HDA in Houdini Apprentice. Include necessary details for this and avoid redundant

information. Describe the whole project objectives (phase 1 and 2, creating materials renders, training network) and focus on implementing materials data from JSON, including render AOVs (passes) we need to get, main specs, etc.

Current state in Houdini (relative screens attached):

Show more ▾

Thought for 1m 0s >

 **Markdown**



New Chat Brief: Build the `neuromat` Material HDA in Houdini Apprentice

I am building `neuromat`, a single master Material HDA for Project Neuron in Houdini Apprentice. This chat should help me finish the HDA and get clean, deterministic material renders for data.

Keep the approach minimal and practical.

Do not redesign the architecture unless there is a **critical** flaw that would break Phase 2

1. Project Objective

Phase 1 – Synthetic Material Dataset

Create a deterministic synthetic render dataset of a single hero asset with many controlled

Current plan:

- one hero model: sculpted Rubber Toy
- one master material HDA: `neuromat`
- materials driven from JSON

- around 2,000 material variations
- around 200 dome cameras
- rendered with stable lighting and fixed scene setup
- output multiple AOVs / passes for later training

The goal of Phase 1 is ****not**** to build a perfect universal material system.

The goal is to generate ****clean, consistent, physically plausible**** renders that will not po

Phase 2 – Train the Material Network

Use the rendered dataset to train the first "Material Hero" network.

High-level target:

- text-conditioned material model
- learns reflectance / appearance from rendered data
- later used inside the broader Neuron pipeline

For Phase 2 to work, Phase 1 must produce data that is:

- deterministic
- consistent across cameras
- consistent across materials
- not dependent on random scene changes
- not contaminated by changing lighting or exposure
- not using camera-space tricks

2. Current State in Houdini

This is the current setup and should remain the working architecture unless there is a criti

Geometry / scene state

- one model: sculpted Rubber Toy
- using UVs to project and place COPS textures
- baked AO and curvature exist on geometry points
- these baked signals are available to drive dirt / wear logic

Shader / material state

- using MaterialX
- using MaterialX Fractal 3D for small noise and other masks
- storing `shader\_parameters` and `procedural\_parameters` on the asset root
- reading those values inside the MaterialX network
- a `VARIATION` map already exists in COPS
- variation map is projected to geometry using UVs

Next material tasks

Need to build and wire:

- `DIRT` map
- `WEAR` map
- `BUMP` map

These maps should be driven by:

- JSON parameters
- existing baked AO
- existing baked curvature
- COPS and MaterialX procedural logic

Render plan

- one material HDA
- JSON-driven material definitions
- later rendered through TOPs
- target scale: 2,000 materials × 200 cameras

3. Architecture Constraints

Do not change these unless there is a critical failure risk.

Keep

- one master material HDA
- one hero geometry
- UV-based projection for current stage
- JSON-driven material parameters
- baked AO and curvature as support signals

- COPS for map generation
- MaterialX for shading
- TOPs for final rendering

Important note about UVs

UV-based projection is acceptable for this stage because:

- we are using one fixed hero mesh
- we need to move quickly
- the immediate goal is material dataset generation on one asset

Do ****not**** force a triplanar / object-space redesign right now unless UV usage creates a cri

The only important rule is:

- no camera-dependent texture behavior
- no view-dependent baked maps
- no random reprojection changes between renders

4. What `neuromat` Must Do

`neuromat` should be a single reusable material HDA that:

1. receives material data from JSON
2. applies base material values
3. generates procedural mask maps
4. combines those masks into a physically plausible MaterialX shader
5. supports predictable debug / AOV output
6. can be rendered repeatedly in batch without surprises

The HDA should be good enough to:

- preview single materials interactively
- validate stress-test materials
- batch render the dataset later

5. JSON Contract

The material JSON already has a clean split and that structure should be preserved.

Each material entry should include:

`metadata`

Identity only:

- material id
- base
- category
- finish
- condition
- optional color name
- semantic label / descriptive text

`shader\_parameters`

Material-facing values:

- `base\_value`
- `base\_color`
- `metalness`
- `specular\_ior`
- `k`
- `specular\_roughness`
- `specular\_anisotropy`
- `subsurface`
- `sss\_color`
- `coat`
- `coat\_roughness`
- `sheen`
- `transmission`
- `transmission\_color`
- `transmission\_depth`
- `transmission\_scatter`
- `transmission\_dispersion`
- `thin\_walled`
- `metallic\_flake`

```
## `procedural_parameters`
```

Mask / texture control:

- `variation\_seed`
- `bump\_scale`
- `bump\_type`
- `noise\_scale`
- `dirt`
- `wear`

The HDA should consume these values directly or through attributes copied onto the asset root

6. Material Logic We Need

The HDA does not need a huge shading system.

It needs a **controlled minimal system** that covers the material library correctly.

```
## Core material families
```

Support at least:

- metals
- dielectrics
- translucent materials
- organics if already defined in JSON

```
## Procedural families
```

Support these procedural branches:

- `stochastic`
- `directional`
- `cellular`
- `none`

Meaning:

- `stochastic`: subtle noise, matte breakup, satin breakup, polished micro-imperfection, dust
- `directional`: brushed / aligned scratch logic
- `cellular`: hammered / crater / pitted logic

- `none`: truly flat procedural branch, use only if intentionally required

7. The Mask System to Build Now

The current next step is to finish the internal signal system.

We need these maps:

7.1 `VARIATION`

Current state:

- already built in COPS
- projected using UVs
- driven by procedural fractal structure

Purpose:

- subtle base color modulation
- subtle roughness breakup
- avoid perfectly flat surfaces

Rule:

- keep it gentle
- it should not become visible dirt or visible wear

7.2 `DIRT`

Need to build.

Purpose:

- broad surface contamination
- accumulation in occluded or sheltered areas
- roughness increase
- optional desaturation / greying / dust tint

Primary drivers:

- baked A0
- large-scale procedural noise

- optional breakup distortion
- JSON `dirt` parameter as intensity control

Expected behavior:

- more accumulation in cavities / concave regions
- broad soft patterns, not scratch detail
- should work for both hard surfaces and translucent materials, but subtly

Use cases:

- dusty car paint
- dusty honey
- light contamination on rubber / metal if required

7.3 `WEAR`

Need to build.

Purpose:

- exposed edges
- abrasion
- scratches / worn zones
- roughness change
- possibly brightening or darkening depending on material family

Primary drivers:

- baked curvature
- medium / high frequency procedural breakup
- JSON `wear` parameter

Expected behavior:

- edges and exposed forms respond first
- wear should not look like dirt
- wear should be sharper and more localized than dirt

Use cases:

- scratched iron
- worn rubber
- exposed highlights on hard edges

7.4 `BUMP`

Need to build.

Purpose:

- micro-surface shape
- physically believable breakup
- procedural family differentiation

Driven by:

- `bump\_scale`
- `bump\_type`
- `noise\_scale`
- `variation\_seed`

Branch meaning:

- `stochastic`: fine noise / subtle irregularity
- `directional`: brushed / linear grain
- `cellular`: crater / hammered pattern

Rule:

- keep bump believable
- do not overdrive displacement-like behavior
- this is primarily micro-surface response, not large-scale deformation

8. Minimal Wiring Strategy

Keep the shader logic simple and stable.

Base color

Base color should come from JSON, then optionally be modified by:

- `VARIATION`
- `DIRT`

Recommended effect:

- variation = subtle value / saturation shift
- dirt = contamination tint or grey/dust bias

Roughness

Roughness should come from JSON, then be modified by:

- `VARIATION` slightly
- `DIRT` strongly in contaminated areas
- `WEAR` where abrasion should alter micro-surface response
- `BUMP` should affect shading physically through normal/bump, not fake roughness only

Metalness / IOR / transmission

These should remain mostly direct JSON-driven values.

Do not procedurally randomize them.

Coat

Use coat exactly where JSON says so.

Do not invent secondary coat logic unless required.

Transmission / translucent materials

Keep this controlled and simple.

The important rule is consistency, not physical overcomplication.

9. Required Render Outputs / AOVs

The dataset must include the right passes for later training.

These are the required passes.

9.1 Beauty

Final shaded render.

Purpose:

- main target image for training
- includes full material and lighting response

9.2 Alpha / Mask

Binary object coverage.

Purpose:

- object silhouette
- background separation
- occupancy / density support

9.3 Depth / `Pz`

Camera depth.

Purpose:

- spatial cue
- helps later training / debugging
- useful for geometry-aware supervision

9.4 BaseColor

Raw material base color input.

Purpose:

- material identity signal
- decouples raw albedo from shaded beauty
- especially important for metal / dielectric interpretation

Important:

- do not let this become black for materials that still need a meaningful material-color sig

9.5 Roughness

Raw roughness input.

Purpose:

- supervision for micro-surface behavior
- helps network separate color from reflectance structure

9.6 Normals

World-space normals.

Purpose:

- geometric orientation cue
- helps network understand surface direction relative to light and camera

9.7 World Position / `P`

World-space position.

Purpose:

- explicit spatial context
- very useful for coordinate-based learning and debugging

10. Optional Debug / Utility Passes

These are not mandatory for the first dataset, but are useful while building `neuromat`.

Useful debug passes

- AO
- Curvature
- Variation map
- Dirt map
- Wear map
- Bump strength / preview
- Material ID
- Camera ID

These help validate the material pipeline before launching batch renders.

Use them for lookdev and QA, not necessarily for the final training set unless needed.

11. Render Specs

Keep render specs simple and stable.

Scene rules

- fixed hero geometry
- fixed geometry scale
- fixed lighting rig
- fixed exposure
- fixed color management settings
- fixed background
- no random environment changes between materials

Camera rules

- 200 cameras on a dome
- same focal setup unless there is a deliberate reason to vary it
- same render settings for every material and every camera

Lighting rules

Lighting must be identical across the whole dataset.

Why:

- the network must learn material behavior, not environment changes

Allowed:

- one fixed HDRI / dome
- optional fixed neutral rim / key setup

Not allowed:

- per-material light randomization
- exposure drift
- camera-dependent light changes

Output format

Use linear formats appropriate for training.

Prefer consistency over experimentation.

General rule:

- do not bake tone-mapped beauty as the only source of truth
- preserve linear render data where possible

12. Dataset Manifest / Metadata Requirements

The render dataset step comes after `neuromat`, but the HDA should be built with it in mind.

Each rendered frame should later be traceable to:

- material id
- semantic label
- JSON parameters snapshot or reference
- camera id
- camera transform
- camera intrinsics
- file paths for all passes

A later `transforms.json` / manifest should be able to connect:

- image files
- camera matrices
- material metadata
- semantic text labels

13. What Must Not Go Wrong

These are the real failure cases for Phase 2.

Avoid these:

Critical failures

- non-deterministic mask generation
- lighting changes across dataset
- exposure drift
- camera-space procedural tricks
- material values changing outside JSON control
- passes that do not match the beauty render

- mislabeled AOVs
- procedural masks so strong that they override the material identity
- UV artifacts severe enough to create obvious training noise

Acceptable limitations for now

- one hero asset only
- UV-based texturing
- not fully ideal physical shading
- not fully generalized object-space material projection
- simple procedural logic as long as it is deterministic and visually coherent

14. Build Order for This Chat

In this new chat, help me in this order:

Step 1

Review the `neuromat` HDA structure and define the minimal parameter interface.

Step 2

Design the internal logic for:

- `VARIATION`
- `DIRT`
- `WEAR`
- `BUMP`

using:

- JSON controls
- AO
- curvature
- COPS
- MaterialX procedural textures

Step 3

Wire these masks into a stable MaterialX shader:

- base color

- roughness
- bump / normal
- transmission / coat where needed

Step 4

Define the required AOVs and how to extract them cleanly in Solaris / Karma.

Step 5

Prepare the setup so it can later be rendered through TOPs for:

- 2,000 material variations
- 200 cameras

Do not jump to training code yet.
First finish `neuromat` correctly.

15. Working Philosophy

I am a solo artist.
This project must move forward.
Perfection is not the goal.

The correct approach is:

- keep it minimal
- keep it deterministic
- keep it physically plausible
- avoid critical mistakes
- accept non-ideal choices if they do not break Phase 2

I do ****not**** want a big theoretical redesign.
I want a practical path to finish `neuromat`, get good renders, and move to dataset generati

16. Expected Help Style

In this chat, help me with:

- concrete HDA design decisions
- MaterialX wiring logic
- mask-building logic in COPS / MaterialX
- pass / AOV planning
- practical Houdini-specific implementation steps
- debugging suspicious material behavior

Avoid:

- abstract research detours
- large architecture rewrites
- advice that assumes a team or long pipeline schedule
- anything that ignores the current working setup

Start from the current architecture and push it to a clean usable state.



Fri, Apr 10 at 2:55 PM

What do you think about this:

""""

C. Expand Prompt Augmentation

In BuildPrompts._build_label, you have the line:

rng.choice([template]). Currently, this doesn't do much since it's picking from a list of one item.

Recommendation: When training the Material Hero INR, the CLIP text-conditioning will generalize better if it sees variations in sentence structure. Instead of a single

[Show more](#) 

Thought for 19s >

Yes, it makes sense.

The idea is good, but keep it controlled.

More prompt variation can help Phase 2 text-conditioning, but only if the variations:

- preserve the same semantic meaning
- do not introduce new material facts
- do not drift into different camera / framing / style language
- remain deterministic for the same material id if needed

You do **not** want "prompt creativity."

You want **syntactic variety with semantic stability**.

 **Markdown**



```
# Developer Brief: Controlled Prompt Augmentation for `BuildPrompts`
```

```
## Goal
```

```
Improve prompt diversity for Phase 2 text-conditioning without weakening label consistency.
```

```
The purpose of this change is to expose the future Material Hero network to small variations
```

```
This is a **controlled augmentation** task, not a creative-writing task.
```

```
---
```

```
# 1. Why This Change Makes Sense
```

The current logic uses effectively one label structure per semantic mode.

That is too rigid.

If the text-conditioning model always sees the same sentence form, it may overfit to wording

Small prompt variation is useful because it teaches the model that these mean the same thing

- close-up of polished gold
- polished gold material
- texture study of polished gold

This improves robustness.

But the variation must stay narrow and disciplined.

2. Core Rule

Only vary ****syntax****, not ****meaning****.

The prompt variants must keep these facts unchanged:

- base material identity
- finish identity
- condition identity
- contamination / abrasion meaning
- translucent / frosted meaning
- relative material emphasis

Do not introduce wording that changes:

- camera distance significantly
- render style
- artistic mood
- lighting setup

- scale implication
- object type

3. What to Change

Current problem

There is effectively one template path per mode, or a random choice over a single-item list.

That adds no real diversity.

Required change

For each semantic mode, define a small set of equivalent templates.

Recommended:

- 3 templates for `pristine`
- 3 templates for `abrasion`
- 3 templates for `contamination`
- 2 or 3 templates for translucent / frosted special cases if needed

Keep the number small.

Do not create 20 variants.

4. Recommended Template Strategy

4.1 Pristine mode

These should emphasize:

- base material
- finish
- clean intact state

Examples of acceptable structure types:

- close-up of X with Y, in pristine condition
- photorealistic material study of X with Y, clean and intact

- texture study of X with Y, in pristine condition

4.2 Abrasion mode

These should emphasize:

- the original finish
- interruption by scratches / abrasion / wear

Examples of acceptable structure types:

- study of X with Y, showing visible abrasion
- close-up of X with Y disrupted by scratches
- photorealistic material study of X, with Y and visible surface wear

4.3 Contamination mode

These should emphasize:

- base finish first
- contamination as surface layer
- no duplicated dust wording

Examples of acceptable structure types:

- study of X with Y and light surface contamination
- close-up of X with fine dust settled on the surface
- photorealistic material study of X with Y, showing surface dust residue

4.4 Translucent / frosted special handling

These should preserve the key semantic distinction:

- clear transmissive
- frosted / blurred transmission
- not generic volumetric unless explicitly intended

Examples of acceptable structure types:

- close-up of X with a frosted surface that blurs transmitted light
- study of X with rough translucent transmission
- photorealistic material study of X with clear refractive transmission

5. Important Restrictions

Do not introduce style drift

Avoid words like:

- cinematic
- dramatic
- studio-lit
- macro photography
- ultra-detailed
- game-ready
- realistic lighting
- glossy render

Unless those ideas are already part of the stable semantic system, they should not appear.

Why:

They introduce new learning signals unrelated to material identity.

Do not introduce framing drift

Avoid switching between:

- close-up
- macro
- texture tile
- full object
- surface patch

Pick a narrow vocabulary and keep it stable.

Recommended allowed framing language:

- close-up
- material study
- texture study

That is enough.

Do not introduce synonyms that change physics

Avoid uncontrolled swapping like:

- polished ↔ glossy

- frosted ↔ cloudy
- scratched ↔ damaged
- dusty ↔ dirty

Only use synonym changes if they are explicitly authored and verified to preserve meaning.

6. Determinism Policy

This must remain deterministic.

Required rule

Template choice should stay reproducible for the same run settings.

Use the existing seeded RNG flow.

Do not use uncontrolled randomness.

Recommended behavior

Given:

- same library
- same seed
- same material order

the generated labels should remain stable.

That gives:

- reproducible training data
- reproducible debugging
- controlled comparisons between runs

7. Implementation Design

7.1 Replace single-template mode with template families

Instead of one template string per mode, define template lists per semantic mode.

Suggested structure:

- pristine templates
- abrasion templates
- contamination templates
- translucent/frosted overrides if needed

7.2 Keep template selection mode-aware

The template family should be selected based on:

- `condition\_mode`
- optional material class overrides
- optional frosted/translucent flags

Do not randomly mix all templates across all modes.

7.3 Keep the semantic composition first

The pipeline should remain:

1. derive semantic parts
2. choose appropriate template family
3. choose one template from that family
4. assemble final label
5. validate / clean formatting

Do not let template choice happen before semantic composition.

8. Suggested Scope of Variation

Keep variation shallow.

Good variations:

- "A close-up of X with Y."
- "Photorealistic study of X with Y."
- "Texture study of X with Y."

Bad variations:

- "A dramatic macro shot of X under studio lighting..."
- "An artistic rendering of..."
- "A detailed hero material featuring..."

These add unrelated concepts.

9. Update Validation

This change should include stronger validation for generated labels.

Add checks for:

- duplicated phrases
- repeated contamination wording
- malformed punctuation
- semantic contradictions
- empty assembled labels

Add optional QA check:

Ensure multiple templates for the same material still preserve the same core tokens:

- base concept
- finish concept
- condition concept

10. Recommended Constraints for the Developer

Keep template counts small

Do not exceed about:

- 3 to 4 templates per mode

More is not better here.

Prefer consistency over variety

The goal is not natural-language richness.
The goal is robust but stable conditioning.

Reuse controlled vocabulary

Stay within the existing semantic system:

- base phrase
- finish description
- condition phrase
- frosted / translucent phrases
- abrasion / contamination phrases

Do not invent new descriptive layers during template selection.

11. Example Design Policy

Use this policy:

- one semantic meaning
- several sentence shells
- same factual content
- deterministic selection
- same cleanup / validation rules

That is the correct level of augmentation.

12. Acceptance Criteria

This change is done when all of the following are true:

- template choice is no longer effectively single-template
- prompt wording varies slightly across generated materials
- semantic meaning remains unchanged
- labels remain deterministic under the same seed

- no prompt introduces unrelated style / framing / lighting concepts
- contamination and abrasion labels remain clean and non-redundant
- translucent / frosted labels preserve their intended physics vocabulary

13. Final Direction

Implement controlled template families.

Do not make the prompts more creative.

Make them slightly more structurally diverse while keeping the meaning locked.

That will help text-conditioning more than a single rigid sentence pattern, without polluting



ChatGPT can make mistakes. Check important info.